El módulo de Mercado (que podríamos llamar "Mercado Negro" o "Paddock Trade") es la zona de mayor riesgo transaccional, pero también el núcleo de la comunidad del juego.

La clave es demostrar que el sistema es inmune a la duplicación de activos (duping) y que la interfaz de usuario guía al jugador sin ambigüedades.

intercambios 1 a 1.

Diseño de la Interfaz (Frontend - React)

La vista MarketView.jsx se dividirá lógicamente en dos paneles o pestañas:

Pestaña A: Crear Oferta. El usuario ve su inventario de "Duplicados". Selecciona uno (Ej: Fiat Uno 1.4s Confort). Luego, se abre un modal con el catálogo completo en gris (siluetas) donde selecciona qué auto pide a cambio.

Pestaña B: Ofertas Globales. Un feed de tarjetas de intercambio. Cada tarjeta muestra visualmente:

Avatar del Usuario que ofrece.

Ofrece: Imagen pequeña del vehículo.

Busca: Imagen pequeña del vehículo solicitado.

Botón de Acción: Si el jugador activo tiene el auto que se busca, el botón dice "Aceptar Trato". Si no lo tiene, el botón está deshabilitado en gris ("Faltan requisitos").

Componente Visual (TradeCard.jsx)

```
export default function TradeCard({ trade, currentUserInventory, acceptTrade }) {

  // Verificamos si el usuario actual tiene el auto que el creador de la oferta está pidiendo

  const hasRequiredVehicle = currentUserInventory.some(

    v => v.id === trade.request_vehicle_id && v.unpasted_count > 0

  );



  return (

    <div className="flex justify-between items-center bg-zinc-800 p-4 rounded-lg border border-zinc-700">

      <div className="flex flex-col items-center">
```

```
        <span className="text-xs text-gray-400">Ofrece</span>
        <img src={trade.offer_image} alt="Ofrecido" className="w-16 h-16 object-contain" />
      </div>

      <div className="text-yellow-400 font-bold text-xl">↔</div>

      <div className="flex flex-col items-center">
        <span className="text-xs text-gray-400">Busca</span>
        <img src={trade.request_image} alt="Buscado" className="w-16 h-16 object-contain opacity-50" />
      </div>

      <button
        disabled={!hasRequiredVehicle}
        onClick={() => acceptTrade(trade.id)}
        className={`px-4 py-2 font-bold rounded ${hasRequiredVehicle ? 'bg-green-500 hover:bg-green-400 text-black' : 'bg-gray-600 text-gray-400 cursor-not-allowed'}`}
      >
        {hasRequiredVehicle ? 'Aceptar Trato' : 'No lo tienes'}
      </button>
    </div>
  );
}
```

2. La Lógica de Backend (FastAPI + SQLite)

El backend debe exponer dos endpoints críticos. En ambos, la integridad de la base de datos es la prioridad.

Endpoint A: Crear la Oferta (POST /api/trades/create)

Antes de insertar la fila en la tabla trades, el servidor debe validar que el usuario no esté publicando un auto que ya pegó o que ya tiene comprometido en otra oferta.

1. Validación de Inventario: ¿Tiene el user_id el offer_vehicle_id con unpasted_count > 0?

2. Validación de Bloqueo Logístico: Para evitar que ofrezca el mismo auto en 5 ofertas distintas, el sistema debe restar temporalmente ese vehículo de su unpasted_count y pasarlo a una columna llamada locked_in_trade (o descontarlo directamente, y si cancela la oferta, se le devuelve). Para el MVP, descontarlo temporalmente es la técnica más segura.

Endpoint B: Ejecutar el Trueque (POST /api/trades/accept/{trade_id})

Aquí es donde brillas frente al jurado. Esta función debe ser una Transacción ACID estricta. Como configuramos SQLite, el motor se encarga de aislar la transacción.

Python:

```
from fastapi import HTTPException

from sqlalchemy.orm import Session

from models import Trade, UserInventory


def execute_trade_transaction(db: Session, trade_id: str, acceptor_user_id: str):

    try:

        # 1. Obtener la oferta y verificar que siga abierta

        trade = db.query(Trade).filter(Trade.id == trade_id, Trade.status == 'OPEN').first()

        if not trade:

            raise HTTPException(status_code=400, detail="La oferta ya no está disponible.")


        # 2. Verificar que el 'acceptor' (quien acepta) tenga la figurita que el creador pide

        acceptor_item = db.query(UserInventory).filter_by(

            user_id=acceptor_user_id,

            vehicle_id=trade.request_vehicle_id

        ).first()
```

```
    if not acceptor_item or acceptor_item.unpasted_count < 1:
        raise HTTPException(status_code=400, detail="No tienes la figurita requerida.")

    creator_user_id = trade.offer_user_id

    # --- INICIO DEL INTERCAMBIO (Mutación de Inventarios) ---

    # 3. Restar el auto a quien acepta
    acceptor_item.unpasted_count -= 1

    # 4. Sumar el auto pedido al creador de la oferta
    creator_receives = db.query(UserInventory).filter_by(user_id=creator_user_id,
vehicle_id=trade.request_vehicle_id).first()
    if creator_receives:
        creator_receives.unpasted_count += 1
    else:
        db.add(UserInventory(user_id=creator_user_id, vehicle_id=trade.request_vehicle_id,
unpasted_count=1))

    # 5. Sumar el auto ofrecido a quien acepta
    acceptor_receives = db.query(UserInventory).filter_by(user_id=acceptor_user_id,
vehicle_id=trade.offer_vehicle_id).first()
    if acceptor_receives:
        acceptor_receives.unpasted_count += 1
    else:
        db.add(UserInventory(user_id=acceptor_user_id, vehicle_id=trade.offer_vehicle_id,
unpasted_count=1))
```

(Nota: El auto del creador ya fue restado al momento de crear la oferta, por lo que no se resta aquí)

6. Cerrar la oferta

trade.status = 'COMPLETED'

7. COMMIT ATÓMICO (Si algo falla antes de esta línea, nada se guarda)

db.commit()

return {"status": "success", "message": "Trueque realizado con éxito"}

except Exception as e:

db.rollback()

raise HTTPException(status_code=500, detail="Error transaccional, trueque cancelado.")

//////////////——————-/////////////////——————-///////////

Manejo en Tiempo Real (Opcional para el MVP)

Para evitar frustraciones donde un usuario hace clic en "Aceptar" y recibe un error de "Oferta ya no disponible" (porque alguien más la aceptó 1 segundo antes), puedes hacer dos cosas:

1. Sencilla (MVP): Capturar el error 400 y mostrar un Toast en rojo: "Alguien fue más rápido, el trato se cerró", y refrescar la lista.

2. Avanzada: Usar Polling en React. Hacer que MarketView.jsx ejecute un fetch a /api/trades/active cada 5 segundos para mantener la lista actualizada automáticamente.